spinalSynth: Implementation Specification

Table of Contents

1. Repository structure

The spinalHDL implementation shall use the following hierarchy for folders, subfolders and files.

```
spinalSynth/
├── build.sbt                  # SBT build configuration (dependencies, Scala version)
├── project/                   # SBT plumbing
│   └── build.properties       # Defines the SBT version
├── src/
│   ├── main/
│   │   └── scala/
│   │       └── synth/         # Root package for the project
│   │           ├── Synth.scala             # Top-level System Integration (per Section
2)
│   │           ├── Main.scala              # Hardware generation entry point (Verilog
generator)
│   │           ├── common/                 # Shared system types and bundles
│   │           │   ├── Types.scala         # Unified hardware types and bundles
│   │           │   └── RomData.scala       # Centralized compiler-time ROM datasets
│   │           ├── timing/                 # System control and tick generation
│   │           │   └── TimingGenerator.scala # Tick generation logic (per Section 3)
│   │           ├── uart/                   # Control Path logic
│   │           │   ├── Uart.scala               # UART Subsystem Wrapper
│   │           │   ├── UartRx.scala             # UART Receiver
│   │           │   ├── UartProtocolDecoder.scala # Protocol Parser
│   │           │   └── RegisterBank.scala       # Parameter Storage
│   │           ├── oscillator/             # Core Oscillator logic (per Section 4)
│   │           │   ├── Oscillator.scala    # Main Oscillator module
│   │           │   ├── Accumulator.scala   # Phase logic
│   │           │   ├── Noise.scala         # LFSR logic
│   │           │   ├── Generators.scala    # Waveform logic (Saw, Tri, etc.)
│   │           │   └── Mux.scala           # Waveform selection
│   │           ├── envelope/               # Envelope Generator logic
│   │           │   ├── EnvelopeGenerator.scala   # Coordinator wrapper module
│   │           │   ├── EnvelopeCtrl.scala        # Control FSM & LUT logic
│   │           │   ├── EnvelopeAccumulator.scala # Phase accumulator logic
│   │           │   └── EnvelopeShaper.scala      # Waveshaper & interpolation logic
│   │           ├── mixing/                 # Audio processing
│   │           │   └── Attenuator.scala    # Volume Control module (per Section 8)
│   │           ├── filter/                 # State Variable Filter (SVF) (per Section
8)
│   │           │   ├── SVF.scala           # Filter Top-Level wrapper
│   │           │   ├── ParameterMapper.scala # Exp and quadratic LUT maps
│   │           │   ├── FilterCore.scala    # FSM arithmetic sequencer
│   │           │   └── FilterMux.scala     # Output selection & saturation
│   │           ├── output/                 # Audio output pipeline
│   │           │   ├── Decimator.scala     # 10x downsampling (per Section 5)
│   │           │   └── I2STransmitter.scala # I2S protocol engine (per Section 6)
```

```
│   └── test/
│       └── scala/
│           └── synth/          # SpinalSim testbenches
│               ├── SynthSim.scala          # Full system simulation
│               ├── timing/
│               │   └── TimingSim.scala     # Verifying tick precision
│               ├── uart/
│               │   ├── RegisterBankSim.scala        # Verifying parameter storage
│               │   └── UartProtocolDecoderSim.scala # Verifying protocol parsing
│               ├── oscillator/
│               │   └── WaveformSim.scala   # Verifying Generator math
│               ├── envelope/
│               │   ├── EnvelopeAccumulatorSim.scala # Verifying accumulator phase &
wrapping
│               │   ├── EnvelopeCtrlSim.scala        # Verifying control FSM & loops
│               │   ├── EnvelopeShaperSim.scala      # Verifying waveshaping & sustain
│               │   └── EnvelopeGeneratorSim.scala   # Verifying full integration ADSR
│               ├── mixing/
│               │   └── AttenuatorSim.scala # Verifying Attenuator scaling
│               └── output/
│                   └── I2STransmitterSim.scala # Verifying I2S timing/protocol
├── rtl/                        # Output folder for generated Verilog/VHDL files
├── doc/                        # Architecture diagrams and design assets
├── README.md                   # Project overview (Context File)
└── Implementation_specs.md     # Technical specification (Context File)
```

---

2. Synth Module (Toplevel)

The Synth module is the hardware entry point and system integration entity.

The module shall:

- Instantiate unified UART Subsystem Wrapper (`Uart`)
- Instantiate the Synthesis, Modulation & Mixing Engine:
  - `TimingGenerator`: Generates clock-enable heartbeats (`phaseTick` and `sampleTick`).
  - `Oscillator`: Core DDS multi-waveform generation.
  - `EnvelopeGenerator`: Core dynamic ADSR waveshaper.
  - `envAttenuator` (10-bit `Attenuator`): Modulates sample volume dynamically via the envelope generator output. Features envelope bypass logic: if envelope bypass (`ENV_CTRL` bit 1) is `1`, a constant maximum `1023` volume is applied.
  - `attenuator` (8-bit `Attenuator`): Manages final master volume scaling.
  - `Decimator`: Downsamples the 480 kHz audio sample stream to 48 kHz.
  - `I2STransmitter`: Serializes parallel audio samples into a stereo I2S bitstream.
- **Clock & Reset Distribution**: Run all submodules synchronously inside a single 24 MHz clocking area with asynchronous active-high reset active.
- **Natural Pipeline Delay & Flow Handshaking**: All audio modules interface using `Flow` (valid/payload) handshakes. Signals propagate timing information naturally. The `svf` module's FSM sequencer is driven directly by `timingGen.io.phaseTick` (480 kHz), and the `Decimator` downsampling is driven directly by `timingGen.io.sampleTick` (48 kHz) without requiring artificial alignment delay registers.
- **Subsystem Isolation**: Keep Synth strictly as an abstraction/integration layer containing no DSP math details or physical protocol implementation details.

Clocking and Reset

The system operates within a single clock domain managed at the top level:

- **Clock**: 24 MHz external input.
- **Reset**: Asynchronous, Active-High.

IO Bundle

```
val io = new Bundle {
    val clk24MHz = in Bool()
    val reset    = in Bool()

    val uartRx   = in Bool()

    val i2sBclk  = out Bool()
    val i2sLrclk = out Bool()
    val i2sData  = out Bool()
}
```

---

3. TimingGenerator

IO Bundle

```
val io = new Bundle {

    val phaseTick  = out Bool()
    val sampleTick = out Bool()
}
```

Internal Structure

Signal	Width	Purpose
`phaseCounter`	6 bit	modulo-50 counter
`sampleCounter`	9 bit	modulo-500 counter

Both counters:

- operate directly from the 24 MHz master clock
- run independently
- are free-running

Tick Behaviour

Both tick outputs:

- are registered
- synchronous
- one clock cycle wide

- default to `False`

---

4. Uart & Registers

The UART package encapsulates serial data decoding, command protocol framing, and parameter register storage.

Package Structure

```
Uart
 ├── UartRx
 ├── UartProtocolDecoder
 └── RegisterBank
```

IO Bundle

```
val io = new Bundle {
    val rx        = in Bool()
    val config    = out(OscillatorConfig())
    val envConfig = out(EnvelopeConfig())
}
```

4.1 UartRx

Purpose: Converts the serial UART bitstream into parallel 8-bit bytes. It operates at 115,200 baud using a 208-tick bit period and includes start-bit verification.

IO Bundle

```
val io = new Bundle {
    val rx      = in Bool()
    val byteOut = master(Flow(Bits(8 bits)))
}
```

4.2 UartProtocolDecoder

Purpose: Frames individual bytes into 3-byte command packets: `[Address/Command]`, `[Data]`, and `[Reserved]`. It asserts `writeEnable` only when a full frame is valid.

IO Bundle

```
val io = new Bundle {
    val rxByte   = slave(Flow(Bits(8 bits)))
    val regWrite = master(Flow(RegisterWrite()))
}
```

4.3 RegisterBank

Purpose: Stores the current state of the synthesizer parameters. It implements an atomic update for the 24-bit frequency word, ensuring all three bytes are applied simultaneously to the DDS engine upon writing to the high-byte address.

Atomic Write Staging Mechanism

To prevent audibly jarring sweep glitches or frequency artifacts, the multi-byte frequency configuration transitions atomically:

- **`FREQ_LOW` (`0x00`)**: Staged into a temporary staging register `freqLowShadow`.
- **`FREQ_MID` (`0x01`)**: Staged into a temporary staging register `freqMidShadow`.
- **`FREQ_HIGH` (`0x02`)**: Directly commits the newly written high byte (`freqHighReg`) and transfers both staged shadow registers (`freqMidReg := freqMidShadow`, `freqLowReg := freqLowShadow`) to the active registers simultaneously on a single clock edge.

IO Bundle

```
val io = new Bundle {
    val regWrite  = slave(Flow(RegisterWrite()))
    val config    = out(OscillatorConfig())
    val envConfig = out(EnvelopeConfig())
}
```

---

5. Oscillator

The Oscillator package connects the four submodules, as shown in the following package structure. It serves as an abstraction layer above the generation of the oscillating audio signals.

Package Structure

```
Oscillator
 ├── Accumulator
 ├── Noise
 ├── Generators
 └── Mux
```

Purpose

Core waveform synthesis engine.

Responsibilities:

- phase accumulation
- waveform generation
- noise generation
- mux between waveforms and noise
- oversampled sample generation

IO Bundle

```
val io = new Bundle {
    val phaseTick = in Bool()
    val config    = in(OscillatorConfig())
    val sample    = master(Flow(SInt(16 bits)))
}
```

5.1 Oscillator Submodules

Accumulator

Responsible for:

- phase register
- phase accumulation
- frequency addition

```
val io = new Bundle {
    val phaseTick = in Bool()
    val freqWord  = in UInt(24 bits)
    val phase     = out UInt(24 bits)
}
```

Noise

Responsible for:

- LFSR register
- feedback logic
- shift/update logic
- noise sample generation

```
val io = new Bundle {
    val phaseTick = in Bool()
    val sample    = out SInt(16 bits)
}
```

Generators

Responsible for:

- saw generation
- square generation
- PWM generation
- triangle generation

```
val io = new Bundle {
    val phase    = in UInt(24 bits)
    val pwmWidth = in UInt(8 bits)
    val waves    = out(Waveforms())
}
```

The module shall contain only combinational logic.

Mux

Responsible for:

- waveform or noise selection
- final waveform routing

```
val io = new Bundle {
    val waveSelect = in UInt(3 bits)
    val waves      = in(Waveforms())
    val noiseWave  = in SInt(16 bits)
    val sample     = out SInt(16 bits)
}
```

5.2 Oscillator signal flow

```
Accumulator ── phase ──┐
                       │
                       ↓
                  Generators
                       │
                       ├── sawWave
                       ├── squareWave
                       ├── pwmWave
                       └── triangleWave

Noise ── noiseSample ──┘
                       ↓
                     Mux
                       ↓
                   sample
```

---

6. Envelope Generator

6.1 EnvelopeGenerator (Top-Level Wrapper)

Purpose

The `EnvelopeGenerator` top-level wrapper acts as the coordinator. It encapsulates the three core submodules (`EnvelopeCtrl`, `EnvelopeAccumulator`, and `EnvelopeShaper`), binds them together, registers parameters to the global register map, and packages the outputs into a synced flow rate.

IO Bundle

```
class EnvelopeGenerator extends Component {
  val io = new Bundle {
    val phaseTick = in Bool()                // 480 kHz audio rate tick
    val syncIn    = in Bool()                // Trigger for Hard Sync
    val config    = in(EnvelopeConfig())     // Packaged register configurations

    val envelopeOut       = master(Flow(UInt(10 bits))) // Unipolar output (0 to 1023)
    val envelopeOutSigned = master(Flow(SInt(10 bits))) // Bipolar output (-512 to
+511)
  }
}
```

Internal Architecture

The wrapper instantiates the submodules and wires their control signals. The outputs from the `EnvelopeShaper` are driven directly to the system audio/control busses.

```
  // Instantiate submodules
  val ctrl        = new EnvelopeCtrl()
  val accumulator = new EnvelopeAccumulator()
  val shaper      = new EnvelopeShaper()

  // Connecting Ctrl to Accumulator
  accumulator.io.resetAccum   := ctrl.io.resetAccum
  accumulator.io.runAccum     := ctrl.io.runAccum
  accumulator.io.accumDir     := ctrl.io.accumDir
  accumulator.io.phaseInc     := ctrl.io.phaseInc
  accumulator.io.sustainLevel := io.config.sustain
  accumulator.io.activeStage  := ctrl.io.activeStage
  ctrl.io.segmentDone         := accumulator.io.segmentDone

  // Connecting Accumulator and Ctrl to Shaper
  shaper.io.phaseTick    := io.phaseTick
  shaper.io.baseIndex    := accumulator.io.baseIndex
  shaper.io.fraction     := accumulator.io.fraction
  shaper.io.curveSelect  := ctrl.io.curveSelect
  shaper.io.activeStage  := ctrl.io.activeStage
  shaper.io.accumDir     := ctrl.io.accumDir

  // Connecting Top-Level inputs to Ctrl
  ctrl.io.syncIn         := io.syncIn
  ctrl.io.config         := io.config

  // Top-Level Outputs connected to Shaper
  io.envelopeOut       <> shaper.io.envelopeOut
  io.envelopeOutSigned <> shaper.io.envelopeOutSigned
```

6.2 EnvelopeCtrl

Purpose

The `EnvelopeCtrl` submodule serves as the "brain". It is responsible for parsing control registers, driving the ADSR playback state machine, computing direction and synchronization triggers, and fetching stage-appropriate increment values from a static lookup table.

IO Bundle

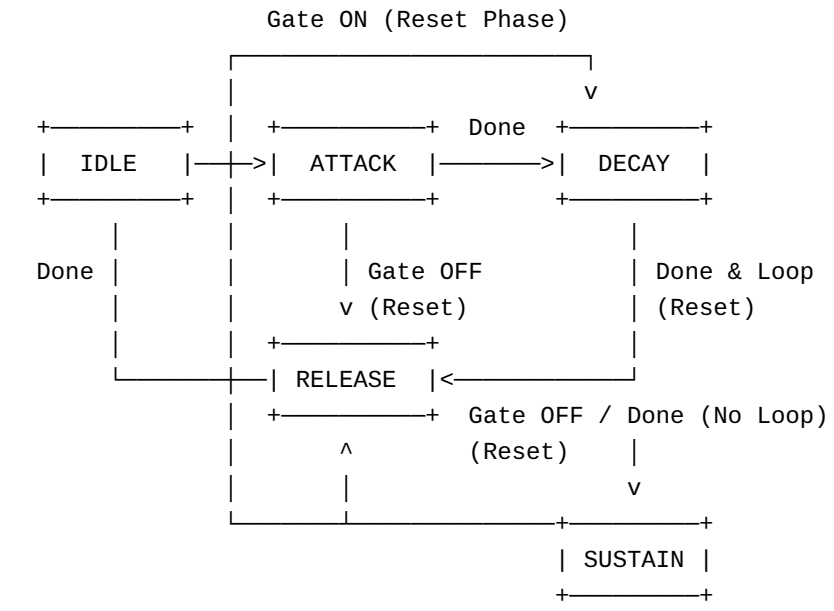
```
class EnvelopeCtrl extends Component {
  val io = new Bundle {
    // Inputs from Top Wrapper
    val syncIn      = in Bool()
    val config      = in(EnvelopeConfig())
    val segmentDone = in Bool()   // High when accumulator sweeps to target limit

    // Outputs to Accumulator
    val resetAccum  = out Bool()
    val runAccum    = out Bool()
    val accumDir    = out Bool()  // 0 = Forward, 1 = Reverse
    val phaseInc    = out UInt(22 bits)

    // Outputs to Shaper
    val curveSelect = out UInt(2 bits)
    val activeStage = out UInt(3 bits) // IDLE=0, ATTACK=1, DECAY=2, SUSTAIN=3,
RELEASE=4
  }
}
```

State Machine Design

The ADSR engine is implemented as a built-in SpinalHDL `StateMachine` (`spinal.lib.fsm`).



It controls the envelope stages using five main states:

- **IDLE (Stage 0):** The default idle state. A `Gate ON` trigger resets the accumulator and transitions the FSM to `ATTACK`.
- **ATTACK (Stage 1):** The phase accumulator counts forward. If `Gate OFF` is detected, it transitions to `RELEASE`. When the segment completes (accumulator hits 1023), it resets the accumulator and transitions to `DECAY`.
- **DECAY (Stage 2):** The phase accumulator counts reverse (downwards). If `Gate OFF` is detected, it transitions to `RELEASE`. When the segment completes (baseIndex counts down to match/cross below `sustainLevel`), it transitions to `SUSTAIN` (or loops back to `ATTACK` if Looping/LFO mode is active).
- **SUSTAIN (Stage 3):** The accumulator is paused, naturally holding the output at the configured sustain level. A `Gate OFF` trigger resets the accumulator and transitions the FSM to `RELEASE`.
- **RELEASE (Stage 4):** The phase accumulator counts reverse (downwards). If `Gate ON` is triggered, it resets the accumulator and transitions to `ATTACK`. When the segment underflows to `0`, it transitions back to `IDLE`.

Playback & Sync Controller

- **Looping (LFO Mode):** When `config.ctrl[2]` (Loop Enable) is active, transitioning out of `DECAY` loops instantly back to `ATTACK` instead of going to `SUSTAIN`.
- **Sync Logic:**
  - **Hard Sync:** A rising edge on `syncIn` forces the FSM back to `ATTACK` and sets `resetAccum := True`.

Logarithmic Time-to-Increment Lookup Table (ROM)

Calculating the logarithmic time-duration mapping at runtime requires expensive divisor blocks. To ensure ASIC portability, the 256 increment coefficients are computed in Scala at compile-time and instantiated as a static hardware ROM (`Mem` in SpinalHDL).

Scala ROM Calculator Formula:

```
val clockFreq = 24000000.0   // 24 MHz
val tMin      = 0.0005       // 0.5 ms
val tMax      = 30.0         // 30.0 s

val lutContent = for (p <- 0 until 256) yield {
  val t = tMin * math.pow(tMax / tMin, p / 255.0)
  val inc = math.round((math.pow(2, 32) / (t * clockFreq)))
  U(inc, 22 bits)
}

val rom = Mem(UInt(22 bits), 256) init(lutContent)
```

6.3 EnvelopeAccumulator

Purpose

The `EnvelopeAccumulator` is a high-speed 32-bit digital register that acts as the phase counter. It increments (or decrements) on every 24 MHz system clock cycle when enabled, driving the envelope's progress through time.

IO Bundle

```
class EnvelopeAccumulator extends Component {
  val io = new Bundle {
    // Inputs from Control Unit
    val resetAccum   = in Bool()
    val runAccum     = in Bool()
    val accumDir     = in Bool()               // 0 = Forward (Up), 1 = Reverse (Down)
    val phaseInc     = in UInt(22 bits)
    val sustainLevel = in UInt(8 bits)
    val activeStage  = in UInt(3 bits)

    // Outputs
    val segmentDone = out Bool()             // Boundary completion pulse
    val baseIndex   = out UInt(8 bits)       // LUT address (Upper 8 integer bits)
    val fraction    = out UInt(2 bits)       // Interpolation fraction (Lower 2
integer bits)
  }
}
```

Counter & Overflow Behavior

- **Accumulator register:** `val accum = Reg(UInt(32 bits)) init(0)`
- **Accumulation Logic:**
  - If `io.resetAccum` is asserted, reset `accum := 0`.
  - If `io.runAccum` is High:
    - If `io.accumDir` is Forward (`False`), `accum := accum + io.phaseInc`.
    - If `io.accumDir` is Reverse (`True`), `accum := accum - io.phaseInc`.
- **Output Splitting:**
  - `io.baseIndex := accum(31 downto 24)`
  - `io.fraction := accum(23 downto 22)`
- **Segment Boundaries & Target Done Detection:**
  - In **Attack (Stage 1)**, completion is hit when the accum register overflows (wraps past 32-bit maximum).
  - In **Decay (Stage 2)**, completion is hit when `baseIndex` counts down to match or cross below `sustainLevel` (`baseIndex <= sustainLevel`).
  - In **Release (Stage 4)**, completion is hit when the accum register underflows (`baseIndex` reaching `0`).
  - `segmentDone := isAttackDone || isDecayDone || isReleaseDone`

6.4 EnvelopeShaper

Purpose

The `EnvelopeShaper` transforms the raw, linear accumulator output into customized, musically natural curves. It reads two consecutive points from a 257-entry curve ROM (Lin, Exp, Log, S-Curve) based on the 8-bit Base Index, performs linear interpolation in pure multiplierless combinational logic using the 2-bit fraction, and outputs unipolar/bipolar audio-rate flows.

IO Bundle

```
class EnvelopeShaper extends Component {
  val io = new Bundle {
    val phaseTick    = in Bool()             // Gating tick
    val baseIndex    = in UInt(8 bits)
    val fraction     = in UInt(2 bits)
    val curveSelect  = in UInt(2 bits)       // 00=Lin, 01=Exp, 10=Log, 11=S-Curve
    val sustainLevel = in UInt(8 bits)
    val activeStage  = in UInt(3 bits)
    val accumDir     = in Bool()

    // Output flows
    val envelopeOut       = master(Flow(UInt(10 bits)))
    val envelopeOutSigned = master(Flow(SInt(10 bits)))
  }
}
```

ROM Curves

The shaper houses four distinct compile-time calculated ROMs, each with **257 entries** to safely compute lookup boundary intervals `LUT[x+1]` when `x = 255` without dynamic wrapping checks.

- **Linear ROM:** ☑ = x * (255.0 / 255.0) (0 to 255)
- **Exponential ROM:** Pre-calculated with decaying inverse-charge capacitor curves.
- **Logarithmic ROM:** Pre-calculated with fast initial rise profiles.
- **S-Curve (Sigmoid) ROM:** Pre-calculated with ease-in/ease-out cosine profiles.

```
val linRom = Mem(UInt(8 bits), 257) init(linContent)
val expRom = Mem(UInt(8 bits), 257) init(expContent)
val logRom = Mem(UInt(8 bits), 257) init(logContent)
val sigRom = Mem(UInt(8 bits), 257) init(sigContent)
```

The implementation of the ROMs moved into the common package and is described in chapter 11.2.

Multiplierless Hybrid 8+2 Linear Interpolation

Linear interpolation requires the formula: `Y = Y0 + (f / 4) * (Y1 - Y0)`. To prevent expensive synthesis of physical multipliers on custom silicon, the fraction multiplication `(f/4) * delta_Y` is resolved in combinational shift-add structures.

- **Direct Fraction Usage**: Since the phase accumulator register naturally counts backwards when `accumDir = 1` during Decay and Release, its fractional output bits (`fraction`) already decrement from 3 to `0`. Consequently, `io.fraction` is fed directly into the interpolation multiplexer in both directions without any modification, ensuring perfectly smooth and monotonic linear transitions.

```
val y0 = UInt(8 bits)
val y1 = UInt(8 bits)

// Multiplexer select based on curveSelect register
y0 := curveMux(io.curveSelect, io.baseIndex)
y1 := curveMux(io.curveSelect, io.baseIndex + 1)

// Safely zero-extend to 9-bit SInt to prevent signed casting MSB sign-bit bugs
val y0Signed = y0.intoSInt
val y1Signed = y1.intoSInt

// Compute signed delta: Y1 - Y0 (10-bit signed SInt)
val delta = y1Signed - y0Signed

// interp holds (Y0 * 4 + f * delta) inside a safe 12-bit signed width to avoid
overflow
val interp = SInt(12 bits)

// Pre-shifted terms for clean hardware synthesis
val y0Shifted    = (y0Signed << 2).resize(12 bits) // Y0 * 4
val deltaShifted = (delta << 1).resize(12 bits)   // 2 * delta
val deltaResized = delta.resize(12 bits)

switch(io.fraction) {
  is(0) { interp := y0Shifted }
  is(1) { interp := y0Shifted + deltaResized }
  is(2) { interp := y0Shifted + deltaShifted }
  is(3) { interp := y0Shifted + deltaShifted + deltaResized }
}

// Convert to 10-bit unsigned unipolar output (0 to 1023)
val finalValUnipolar = interp.asUInt.resize(10 bits)
```

Parallel Bipolar Output Flow:

- **Unipolar Flow:** `io.envelopeOut.payload := finalValUnipolar`
- **Bipolar Flow:** Bypasses arithmetic blocks by shifting the unipolar range (0 to 1023) to signed (-512 to +511) through direct inversion of the unipolar MSB:

  ```
  io.envelopeOutSigned.payload := (finalValUnipolar ^ 0x200).asSInt
  ```

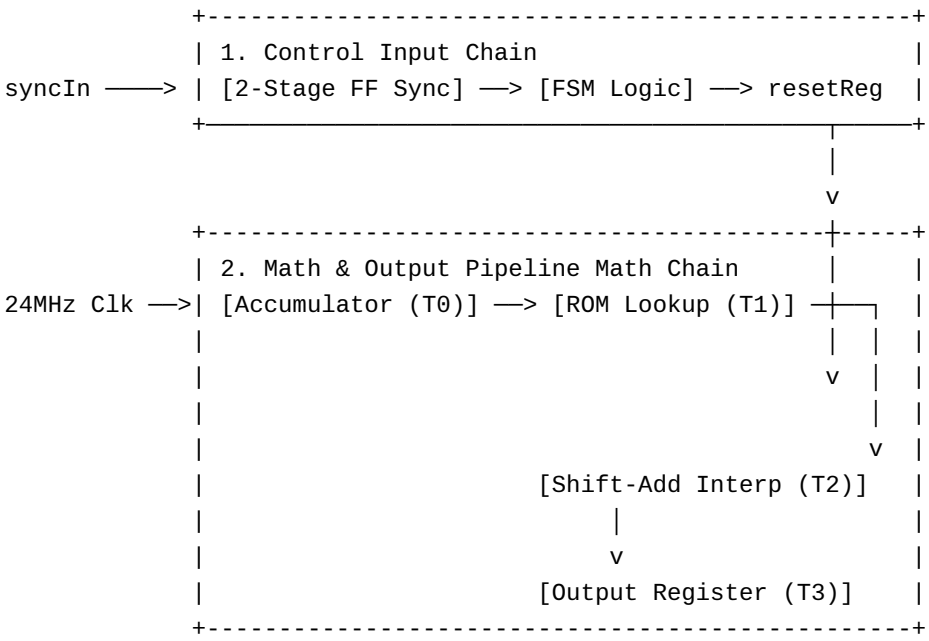
- **Heartbeat Synchronizer:**

  ```
  io.envelopeOut.valid       := io.phaseTick
  io.envelopeOutSigned.valid := io.phaseTick
  ```

6.5 Signal Flow & Pipeline Timing

To ensure hardware timing closure at 24 MHz and robust, glitch-free control transitions, the Envelope Generator architecture isolates data lookup, mathematical computation, and control registers into dedicated synchronous pipeline stages.

The design implements two distinct hardware signal chains:



6.5.1 Chain 1: Control Input Propagation (`syncIn`, `Gate` -> Accumulator)

This path synchronizes asynchronous external controls and propagates internal register signals to steer the accumulator:

- **External Sync Input (`syncIn`):** Connects to a standard 2-stage flip-flop synchronizer clocked at 24 MHz to prevent metastability.
  - *Latency:* **2 clock cycles** (synchronization penalty).
- **Register Settings (`config`):** Synchronous from the `RegisterBank`.
- **State Updates:** The FSM processes inputs combinationally and issues `resetAccum` or `runAccum` control registers to the accumulator on the next cycle.
  - *Total Control Latency:* **3 clock cycles** for `syncIn`, **1 clock cycle** for register-driven gate triggers.

6.5.2 Chain 2: Forward Lookup Math Pipeline (Accumulator -> Output)

This is the core mathematical flow designed to maintain a 24 MHz clock cycle bound through registered cells:

- **T0 (Accumulation Stage):** The 32-bit register `accum` increments (or decrements) by `phaseInc`. The split base index and fraction bits are output stable.
- **T1 (ROM Lookup Stage):** The 8-bit index addresses the dual boundary ports `LUT[x]` and `LUT[x+1]`. The memory array lookup requires **1 clock cycle** to fetch and settle output registers.
- **T2 (Arithmetic Stage):** The combinational multiplexed shift-add interpolator evaluates `Y = Y0 + (f/4) * delta_Y` instantly.
- **T3 (Output Stage):** To isolate critical routing paths, the final unipolar and bipolar amplitude values are registered to the physical output ports and qualified with `phaseTick`.
  - *Total Forward Latency:* **3 clock cycles** (fully balanced).

---

7. Attenuator

Purpose

Applies volume scaling and attenuation dynamically on the oversampled 480 kHz audio sample stream.

Parameterization

The module is parameterized at compile-time to support dynamic input volume register sizes:

- **`volumeWidth`** (`Int`, default = 8): Defines the bit-width of the volume control input.

IO Bundle

```
class Attenuator(volumeWidth: Int = 8) extends Component {
  val io = new Bundle {
    val sampleIn  = slave(Flow(SInt(16 bits)))
    val volume    = in UInt(volumeWidth bits)
    val phaseTick = in Bool()
    val sampleOut = master(Flow(SInt(16 bits)))
  }
}
```

Mathematical Scaling

To prevent sign-bit alignment errors during dynamic multiplication:

1. Zero-extend `io.volume` to a signed integer (`volumeSigned = io.volume.intoSInt`) of size `volumeWidth + 1` bits.
2. Perform signed multiplication between the 16-bit signed input sample and `volumeSigned` to produce a product of size `16 + volumeWidth + 1` bits.
3. Scale the product down by shifting it right by `volumeWidth` bits, and resize back to 16 bits: `scaledSample = (product >> volumeWidth).resize(16 bits)`
4. The payload register (`outReg`) updates on `sampleIn.valid` (1-sample latency).
5. The output `sampleOut.valid` is aligned to the next `phaseTick` edge by delaying the captured valid status by 1 sample using an internal `validReg` updated on `phaseTick` and combinationally gating it with `phaseTick`.

---

8. State Variable Filter (SVF)

8.1 SVF (Top-Level Wrapper)

Purpose

The SVF top-level wrapper acts as the coordinator. It encapsulates the three submodules (`ParameterMapper`, `FilterCore`, and `FilterMux`), binds them together, registers parameters to the global register map, and coordinates the clock-gated flow of audio samples.

IO Bundle

```
class SVF extends Component {
  val io = new Bundle {
    val phaseTick  = in Bool()                // 480 kHz sample rate tick
    val config     = in(FilterConfig())       // Unified register configuration bundle

    val sampleIn   = slave(Flow(SInt(16 bits)))  // 16-bit audio input
    val sampleOut  = master(Flow(SInt(16 bits))) // 16-bit audio output
  }
}
```

Internal Architecture

The top-level wrapper instantiates the submodules, handles parameter conversion, and drives the state variables. When `enable` is deasserted, it clears the internal filter state registers by asserting the `clear` line of `FilterCore` and routes a constant `0` to the output payload, whilst keeping the output flow's `valid` signal pulsing synchronously with the input flow's `valid` (which is gated by `phaseTick`).

8.2 ParameterMapper

Purpose

The `ParameterMapper` converts user-facing 8-bit parameters into high-precision coefficients used by the Chamberlin SVF equations.

IO Bundle

```
class ParameterMapper extends Component {
  val io = new Bundle {
    val cutoff         = in UInt(8 bits)
    val resonance      = in UInt(8 bits)
    val cutoffCoeff    = out UInt(12 bits)
    val resonanceCoeff = out UInt(8 bits)
  }
}
```

Mapping Equations & ROM Tables

- **Cutoff ROM (256 x 12 bits)**: Maps the input exponentially to generate a log-like frequency distribution.

  ```
  cutoffCoeff = round(10.0 * (4095.0 / 10.0)^(p / 255.0))
  ```

- **Resonance ROM (256 x 8 bits)**: Maps the input quadratically, where higher input values represent higher resonance (lower damping coefficient).

  ```
  resonanceCoeff = round(255.0 - 251.0 * (r / 255.0)^2)
  ```

8.3 FilterCore

Purpose

The `FilterCore` implements the core Chamberlin fixed-point arithmetic loops. Rather than implementing parallel arithmetic logic, it uses a **time-multiplexed shared architecture** with exactly **one multiplier** and **one adder/subtractor** to save hardware resource area. The calculation steps are scheduled over multiple clock cycles within the 50-cycle sample frame budget.

IO Bundle

```
class FilterCore extends Component {
  val io = new Bundle {
    val phaseTick      = in Bool()
    val clear          = in Bool()              // Clears states to 0 when filter is
disabled
    val sampleIn       = in SInt(16 bits)
    val cutoffCoeff    = in UInt(12 bits)
    val resonanceCoeff = in UInt(8 bits)

    val lp             = out SInt(24 bits)
    val bp             = out SInt(24 bits)
    val hp             = out SInt(24 bits)
    val done           = out Bool()             // Pulses High when calculation
completes
  }
}
```

Shared Arithmetic Operators

To optimize resource usage:

- **Shared Multiplier**: Performs signed multiplication. Inputs are selected via multiplexers depending on the FSM state. Unsigned coefficients are zero-extended to signed values (`intoSInt`) before multiplication.
- **Shared Adder/Subtractor**: Performs signed addition and subtraction. Inputs are selected via multiplexers depending on the FSM state.

FSM State Scheduling

The sequencing of arithmetic operations is managed using a SpinalHDL `StateMachine` from the `spinal.lib.fsm` library. The FSM cycles through 8 states to compute the complete SVF response:

State	Step / Operation	Multiplier Inputs	Adder Inputs	Registers Updated
`IDLE`	Wait for `phaseTick`	-	-	Latch `sampleIn` on trigger
`CALC_RES`	`resTerm = (bp * resonanceCoeff) >> 8`	`bp`, `resonanceCoeff.intoSInt`	-	`resTerm` (24b)
`SUB_INPUT`	`tempSub = input - lp`	-	`inputExt`, `lp` (Sub)	`tempSub` (24b)

State	Step / Operation	Multiplier Inputs	Adder Inputs	Registers Updated
CALC_HP	hp = tempSub - resTerm	-	tempSub, resTerm (Sub)	hp (24b)
CALC_BP_TERM	bpTerm = (hp * cutoffCoeff) >> 12	hp, cutoffCoeff.intoSInt	-	bpTerm (24b)
UPDATE_BP	bpNext = bp + bpTerm	-	bp, bpTerm (Add)	bp state register
CALC_LP_TERM	lpTerm = (bpNext * cutoffCoeff) >> 12	bpNext, cutoffCoeff.intoSInt	-	lpTerm (24b)
UPDATE_LP	lpNext = lp + lpTerm	-	lp, lpTerm (Add)	lp state register, done

When clear is asserted, FSM resets to IDLE and internal state registers (lp, bp) are cleared to 0.

8.4 FilterMux

Purpose

The FilterMux selects the appropriate response based on the filter mode and scales/resizes the internal 24-bit representation back to the 16-bit output using saturating logic.

IO Bundle

```
class FilterMux extends Component {
  val io = new Bundle {
    val mode      = in UInt(2 bits)
    val lp        = in SInt(24 bits)
    val bp        = in SInt(24 bits)
    val hp        = in SInt(24 bits)
    val sampleOut = out SInt(16 bits)
  }
}
```

Downsizing with Saturation Clamping

Rather than using simple MSB truncation (which leads to severe wrap-around noise when the internal state variables exceed 16-bit limits), FilterMux implements a saturating clamp to bound the output values within the signed 16-bit range:

```
if (selected > 32767) {
    sampleOut = 32767
} else if (selected < -32768) {
    sampleOut = -32768
} else {
    sampleOut = selected.resize(16 bits)
}
```

This preserves the internal 8-bit headroom in the 24-bit FSM registers (`lpReg`, `bpReg`) to keep the filter loop stable, while ensuring output signal peaks clip cleanly (similar to analog saturation) rather than wrapping around.

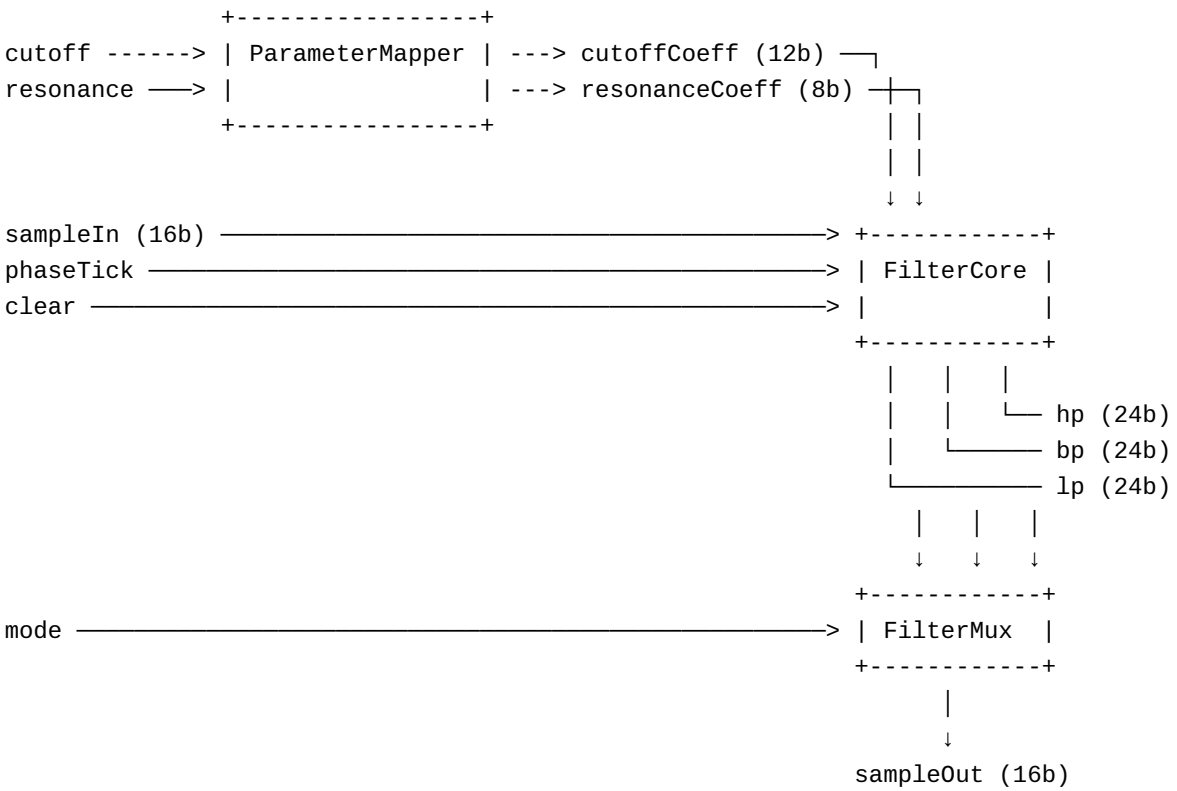
8.5 Register Map Integration

The register space for the Filter Module is allocated at addresses `0x50` to `0x53`:

Register Address	Name	Description
`0x50`	`FILTER_CTRL`	Bit 0: `FILTER_DISABLE` (`0` = active/enabled, `1` = disabled), Bit 1: `FILTER_BYPASS` (`0` = active, `1` = bypassed)
`0x51`	`FILTER_MODE`	Bits 1:0: Response Mode (`00` = LP, `01` = BP, `10` = HP, `11` = Reserved)
`0x52`	`FILTER_CUTOFF`	8-bit user-facing cutoff frequency
`0x53`	`FILTER_RESONANCE`	8-bit user-facing resonance / feedback

The `RegisterBank` will commit updates to these registers on the main clock domain, and outputs will be routed through a `RegNext` synchronization stage to the SVF top-level wrapper inputs.

8.6 Signal Flow & Pipeline Timing



Latency

The filter operations are executed sequentially by the FSM. Once calculations are complete, the resulting sample is held in an intermediate output register.

On the next incoming `phaseTick` pulse, the top-level wrapper asserts the output `sampleOut.valid` and drives the payload. This introduces a latency of exactly **1 sample period** (50 system clock cycles) between the input and output flows. Since the output `valid` is aligned to the 480 kHz `phaseTick` grid, it eliminates fractional clock-cycle offsets and allows clean downsampling downstream.

---

9. Decimator

IO Bundle

```
val io = new Bundle {
    val sampleTick = in Bool()
    val sampleIn   = slave(Flow(SInt(16 bits)))
    val sampleOut  = master(Flow(SInt(16 bits)))
}
```

Responsible for converting the 480 kHz oversampled waveform stream into 48 kHz audio samples. The decimator captures every 10th oscillator sample.

sampleOut is:

- registered
- stable
- updated only at 48 kHz

valid communicates that a new 48 kHz sample is available now.

---

10. I2STransmitter

IO Bundle

```
val io = new Bundle {
    val sampleIn  = slave(Flow(SInt(16 bits)))
    val bclk      = out Bool()
    val lrclk     = out Bool()
    val sdata     = out Bool()
}
```

Self-contained serializer and protocol engine.

Responsibilities:

- BCLK generation
- LRCLK generation
- shift register control
- stereo frame timing
- serial audio output

The module operates directly from the 24 MHz master clock.

The serializer uses the scheduled timing subpattern:

```
16,16,15,16,16,15,16,15
```

to generate the required average I²S bit timing.

10.1 Gated Startup and Idle State

The transmitter employs a gated startup mechanism to ensure that the I2S clock and data lines only toggle when valid audio data is present.

Idle Behavior: The transmitter starts in an inactive state after reset:

- `bclk` and `sdata` are held `Low`.
- `lrclk` is held `High` (the standard idle state for I2S).

Activation: The state machine transitions to `active` upon the first assertion of `io.valid`. On this cycle:

- Internal counters (`bitCounter`, `cycleCounter`, `patternIndex`) are reset to `0`.
- The input sample is latched into the `sampleBuffer`.
- The `shiftRegister` is loaded, and transmission of the Left channel begins immediately.

Once active, the transmitter remains in the active state to maintain a continuous bit clock, even if subsequent `valid` pulses are delayed, though it will re-synchronize its frame boundaries to the `valid` signal to prevent drift.

11. Common Package

The `synth.common` package puts the common goods of the code base into one centralized place. It contains shared types, bundle structures, constants, and pre-calculated ROM lookup arrays. This helps reducing code redundancy and makes the code easier to understand and maintain.

11.1 Types and Configuration Bundles (`Types.scala`)

The `Types.scala` file defines case classes and bundles used for internal bus transactions, component configuration, and inter-module signal routing.

11.1.1 Bus Communication Types

- **`RegisterWrite`**: Represents a decoded write transaction on the control bus.
  - `address`: `UInt(8 bits)` — The target register offset.
  - `data`: `Bits(8 bits)` — The 8-bit parameter payload.

11.1.2 Module Configuration Bundles

To avoid routing cluttered individual control wires throughout the top-level entity, settings from the `RegisterBank` are packaged into unified configuration bundles:

- **`OscillatorConfig`**: Routes parameters from the UART registers to the Oscillator.
  - `freqWord`: `UInt(24 bits)` — Complete committed DDS frequency increment.
  - `waveSelect`: `UInt(3 bits)` — Selection index of the active waveform.
  - `pwmWidth`: `UInt(8 bits)` — Duty cycle for the pulse waveform.
  - `volume`: `UInt(8 bits)` — Target volume setting (Reserved).
- **`EnvelopeConfig`**: Routes ADSR settings to the Envelope Generator.
  - `ctrl`: `Bits(8 bits)` — Bit field controls (`[0]`: Disable, `[1]`: Bypass, `[2]`: Loop,`[3]`: Hard Sync Enable, `[5:4]`: Curve).
  - `attack` / `decay` / `sustain` / `release`: `UInt(8 bits)` — Envelope phase coefficients.
  - `gate`: `Bits(8 bits)` — Bit `[0]`: Gate ON/OFF, Bit `[1]`: Hard Sync Trigger.
- **`FilterConfig`**: Routes parameters to the State Variable Filter.
  - `ctrl`: `Bits(8 bits)` — Bit field controls: (`[0]`: Disable, `[1]`: Bypass).
  - `mode`: `UInt(2 bits)` — Band configuration (`00`=LP, `01`=BP, `10`=HP).
  - `cutoff`: `UInt(8 bits)` — Filter cutoff frequency.
  - `resonance`: `UInt(8 bits)` — Filter resonance feedback strength.

11.1.3 Internal Audio Routing Bundles

- **`Waveforms`**: Routes individual generated waveforms from `Generators` to the output `Mux` combinational logic:
  - `saw` / `square` / `pwm` / `tri`: `SInt(16 bits)`.

11.1.4 Envelope Generator FSM Constants

- **`EnvelopeStage`**: An object defining integer constants representing active FSM phases:
  - `IDLE` = 0, `ATTACK` = 1, `DECAY` = 2, `SUSTAIN` = 3, `RELEASE` = 4.

11.2 Centralized Memory Data (`RomData.scala`)

The `RomData` object centralizes the mathematical modeling and pre-calculated data for all lookup tables in `spinalSynth`. It generates pure Scala sequences (`Seq[BigInt]`), decoupling DSP calculations from physical hardware bit-widths.

11.2.1 Envelope Rate LUT (`envelopeRateLut`)

Generates 256 logarithmic time-duration-to-phase-increment coefficients mapped between 0.5 ms and 30.0 s at a 24 MHz master clock:

```
t(p) = 0.0005 * (30.0 / 0.0005)^(p / 255)
envelopeRateLut(p) = round(2^32 / (t(p) * 24_000_000))
```

11.2.2 Envelope Shaper Curve LUTs (257 entries each)

To support hybrid 8+2 bit interpolation, each curve contains 257 entries, with index 256 acting as the terminal boundary limit to prevent overflow:

- **Linear LUT** (`linearCurveLut`):

  ```
  y(x) = min(255, x)
  ```

- **Exponential LUT** (`expCurveLut`):

  ```
  factor = min(255.0, x) / 255.0
  y(x) = round(255 * (exp(3 * factor) - 1) / (exp(3) - 1))
  ```

- **Logarithmic LUT** (`logCurveLut`):

  ```
  factor = min(255.0, x) / 255.0
  y(x) = round(255 * log1p(7 * factor) / log1p(7))
  ```

- **Sigmoid / S-Curve LUT** (`sigCurveLut`):

  ```
  factor = min(255.0, x) / 255.0
  y(x) = round(255 * (1 - cos(pi * factor)) / 2)
  ```

11.2.3 Filter Coefficient Mapping LUTs (256 entries each)

- **Exponential Cutoff** (`filterCutoffLut`): Maps user-facing cutoff values (0 to 255) exponentially to 12-bit coefficient steps (10 to 4095):

  ```
  k(p) = round(10 * (4095 / 10)^(p / 255))
  ```

- **Quadratic Resonance** (`filterResonanceLut`): Maps resonance values (0 to 255) quadratically to feedback coefficients:

```
q(r) = round(255 - 251 * (r / 255)^2)
```

11.3 Integration in Hardware Modules

Modules instantiate their own local memories for physical routing, but initialize them using `RomData` sequences mapped to hardware types:

```
// In EnvelopeCtrl: Rate LUT
val rom = Mem(UInt(22 bits), 256) init(RomData.envelopeRateLut.map(U(_, 22 bits)))

// In EnvelopeShaper: Curve ROMs
val expRom = Mem(UInt(8 bits), 257) init(RomData.expCurveLut.map(U(_, 8 bits)))

// In ParameterMapper: Cutoff mapping
val cutoffRom = Mem(UInt(12 bits), 256) init(RomData.filterCutoffLut.map(U(_, 12
bits)))
```